Министерство науки и высшего образования РФ
АРЗАМАССКИЙ ПОЛИТЕХНИЧЕСКИЙ ИНСТИТУТ (ФИЛИАЛ)
ФЕДЕРАЛЬНОГО ГОСУДАРСТВЕННОГО БЮДЖЕТНОГО
ОБРАЗОВАТЕЛЬНОГО УЧРЕЖДЕНИЯ ВЫСШЕГО ОБРАЗОВАНИЯ «НИЖЕГОРОДСКИЙ
ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ ИМ. Р.Е.АЛЕКСЕЕВА»
(АПИ НГТУ)

Лабораторная работа №2 по дисциплине «Логика и архитектура вычислительных средств»

Выполнила:
студентка гр. МА26ПМ
Фролова Д.А.

Проверил:
Комаров А.О.

Арзамас, 2026

Сегментация дорожной сцены для беспилотника

1 Цель, требования и ограничения

Цель проекта - принимать видеопоток от БПЛА, нормализовать кадры, выделять дорожное полотно, автомобили, пешеходов, разметку и препятствия, сохранять маски и передавать события в аналитический контур. Результат обработки доступен оператору через API и dashboard. Для сегментации подключается SegFormer с весами Cityscapes; цветовой baseline используется как воспроизводимая локальная точка сравнения. Требования и ограничения проекта представлены в таблице 1.

Таблица 1 - Требования проекта

Категория	Содержание
Функциональные требования	Принимать кадры и видеопоток, выполнять preprocessing, строить маску шести выходных классов, сохранять артефакты, публиковать событие и показывать аналитику.
Входные данные	PNG/JPEG или кадр MP4-потока; frame_id, flight_id, время, EXIF-ориентация; после нормализации RGB и ограничение 1280×720.
Выходные данные	Маска, доли классов, dominant_class, размеры, preprocessing, ссылки на MinIO, событие frames.segmented и dashboard-ready scene_analytics.
Ограничения	Основная количественная оценка выполнена на comma10k с автомобильной камеры; UAVid подключается внешним manifest и требует отдельной загрузки размеченных UAV-кадров.

2 Архитектура и обмен сообщениями

На рисунке 1 показан контейнерный контур приёма кадров и сегментации. API принимает отдельные кадры, VideoStreamAdapter преобразует видеопоток в упорядоченные кадры, а Kafka worker обрабатывает события frames.created. MinIO хранит исходные кадры и маски, PostgreSQL хранит

production-метаданные, а локальная SQLite содержит демонстрационные кадры и scene_analytics. Смена цветового алгоритма на SegFormer задаётся переменной SEGMENTATION_MODEL.

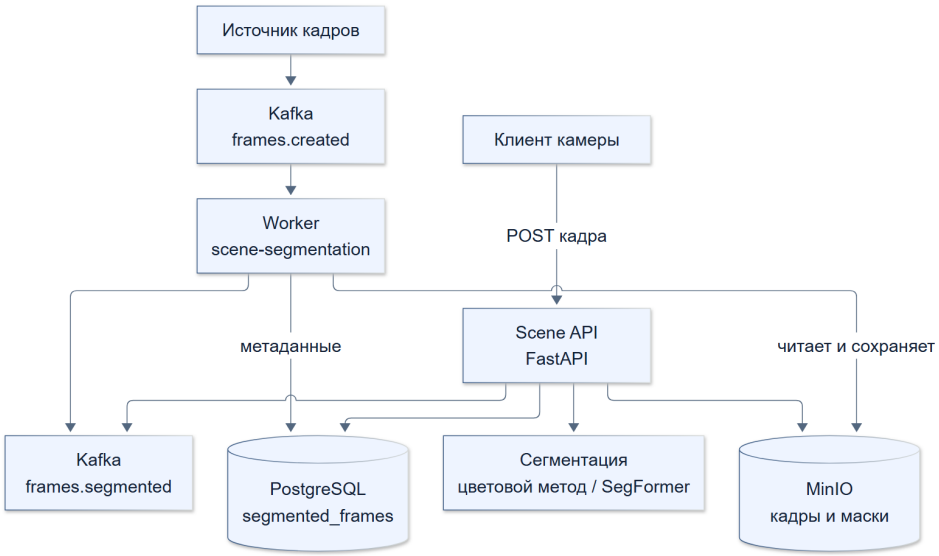


Рисунок 1 - Контейнерная схема сегментация дорожной сцены для беспилотника

Поток обработки показан на рисунке 2. Pipeline разделён на приём видеопотока, preprocessing, выбор стратегии сегментации, построение маски, запись кадров и scene analytics, публикацию Kafka-события и отображение dashboard.

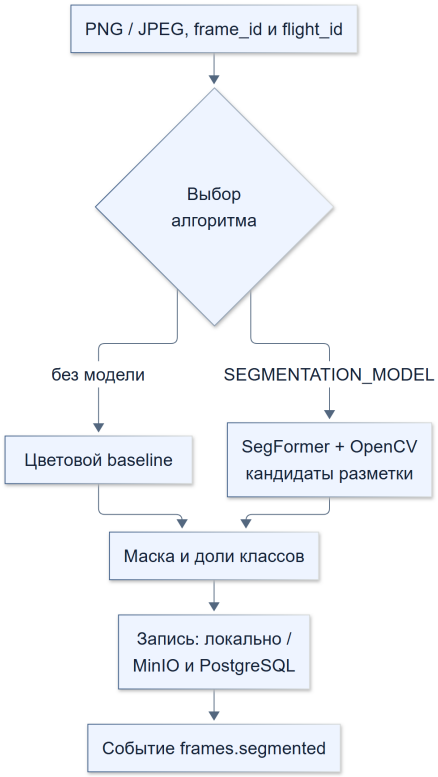


Рисунок 2 - Конвейер обработки данных

Каналы и участники обмена представлены в таблице 2.

Таблица 2 - Каналы взаимодействия

Канал	**Отправитель**	**Получатель**	**Данные**
frames.created	внешний отправитель	scene-segmentation	frame_id, flight_id, bucket, source_object
frames.segmented	API / worker	внешний потребитель	frame_id, маска, class_share
Kafka	broker	consumer group scene-segmentation	UTF-8 JSON

3 Данные, API и хранилища

На рисунке 3 показана концептуальная схема. В SQLite таблица frames имеет первичный ключ frame_id и индекс flight_id/captured_at. Таблица scene_analytics хранит размеры кадра, доминирующий класс и доли шести классов; для неё предусмотрены индексы flight_id/captured_at и dominant_class. PostgreSQL хранит segmented_frames и scene_analytics с первичным ключом frame_id и индексом flight_id/captured_at. Исходные кадры и маски хранятся в MinIO по ключам frames/{frame_id}.png и masks/{frame_id}.png.

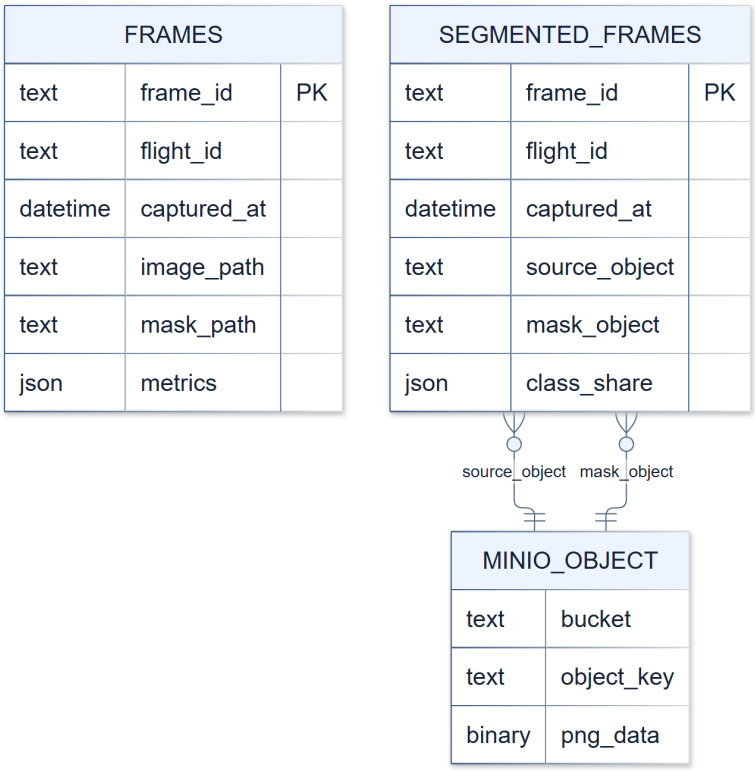


Рисунок 3 - Логическая схема данных

Операции программного интерфейса представлены в таблице 3.

Таблица 3 - Контракт API

Операция	Назначение	Результат
POST /api/v1/frames	Загрузить кадр	201, frame_id и маска
GET /api/v1/frames/{id}	Получить сегментацию	маска и метрики
GET /api/v1/summary	Общая сводка	Число кадров и полётов, доминирующие классы
GET /api/v1/analytics?flight_id={id}	Аналитика полёта	200, список scene_analytics
GET /dashboard и GET /pipeline	Визуальный контроль	HTML dashboard / pipeline

```
POST /api/v1/frames
Заголовки: X-Frame-Id: f1; X-Flight-Id: flight1; Content-Type:
image/png. Тело содержит PNG/JPEG.
Ответ 201 содержит frame_id, flight_id, width, height,
preprocessing, mask, class_share, dominant_class и captured_at.
Дополнительные операции: GET /api/v1/frames/{id}, GET
/api/v1/summary, GET /api/v1/analytics?flight_id=flight1, GET
/dashboard и GET /pipeline.
```

Листинг 1 - Пример запроса к API

4 Паттерны проектирования

Strategy: ColorSegmentationStrategy и ModelSegmentationStrategy предоставляют общий метод predict; segmentation_strategy выбирает обработчик. Adapter: SegFormerAdapter преобразует выход модели в маску. Repository: PostgresFrameRepository и MinioFrameStore изолируют production-хранилища. Pipeline: SegmentationPipeline связывает preprocessing, стратегию и формирование результата. Observer: SceneAnalyticsObserver записывает агрегаты scene_analytics после каждого кадра.

```
Код: store_frame → segmentation_strategy().predict →
SegFormerAdapter или segment → MinioFrameStore и
PostgresFrameRepository (app.py).
```

Листинг 2 - Компоненты обработки кадров

5 Реализация, тестирование и результаты

На 100 автомобильных кадрах comma10k среднее IoU итогового pipeline: дорога 0,7667, разметка 0,3198 и объединённые движущиеся объекты 0,3576. Для исходного класса дороги SegFormer получил 0,7521 против 0,2912 у цветового baseline. Отдельно для дороги/разметки/движущихся объектов сохранены mean-image и pooled IoU. Датасет comma10k снят автомобильной камерой, поэтому качество на БПЛА им не доказывается. Для целевой UAV-проверки добавлена инструкция и manifest для UAVid; внешние файлы датасета не включены из-за лицензии и объёма.

Результаты проверки сценариев приведены в таблице 4.

Таблица 4 - Результаты контрольных сценариев

Сценарий	**Ожидаемый результат**	**Статус**
Кадр дороги	Создана маска дороги и разметки	Проверен
Кадр с пешеходом	Событие содержит класс pedestrian	Пройден на синтетическом кадре (color baseline); реальная отдельная разметка не проверялась
Некорректный файл	API возвращает 422	Проверен
Видеопоток из 8 кадров	Все кадры обработаны и записаны в analytics	Пройден локально

Рисунок 4 показывает документацию FastAPI при локальном запуске. Dashboard и pipeline представлены на рисунках 8 и 9; результаты проверки SegFormer и OpenCV приведены в приложении Б.

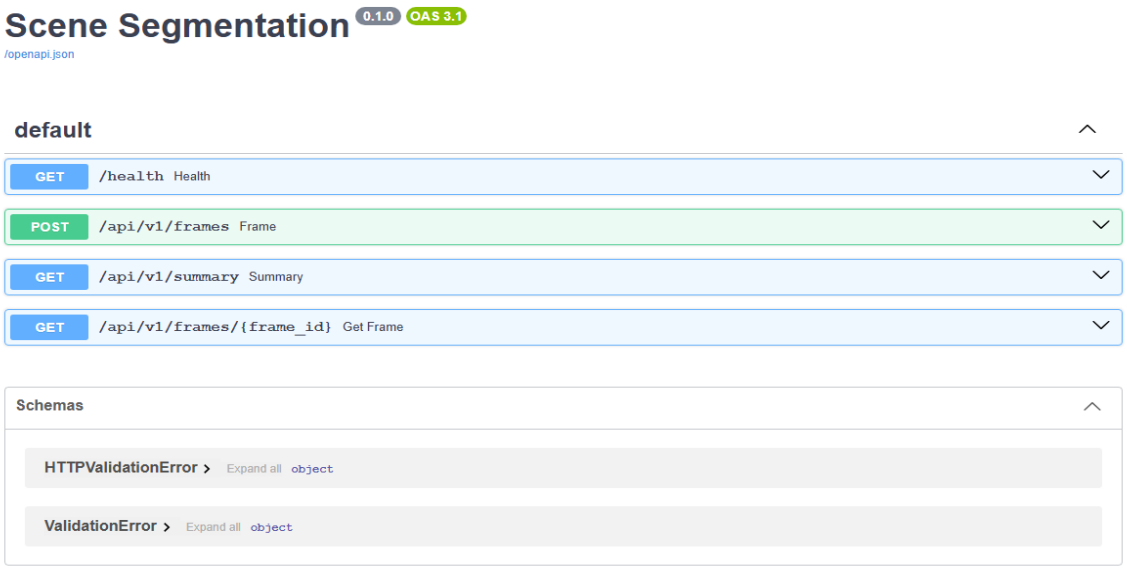


Рисунок 4 - Документация запущенного API

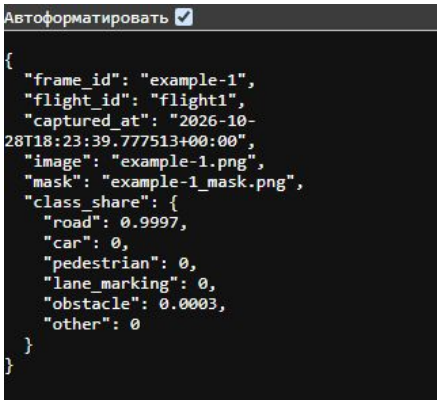


Рисунок 5 - Ответ API после обработки кадра цветовым методом

Проект задаёт расширяемый конвейер семантической сегментации. Preprocessing, Pipeline и Observer выделены в отдельные компоненты; единые HTTP/Kafka-контракты позволяют подключать новые модели и источники видеопотока без изменения клиентского API.

6 Архитектура C4 и границы сервисов

Контекстная схема отделяет оператора БПЛА, клиент видеопотока, Scene Segmentation System, архив и аналитический контур. Компонентная схема показывает VideoStreamAdapter, FramePreprocessor, SegmentationPipeline, две стратегии, MinIO/PostgreSQL Repository, SceneAnalyticsObserver и Kafka publisher.

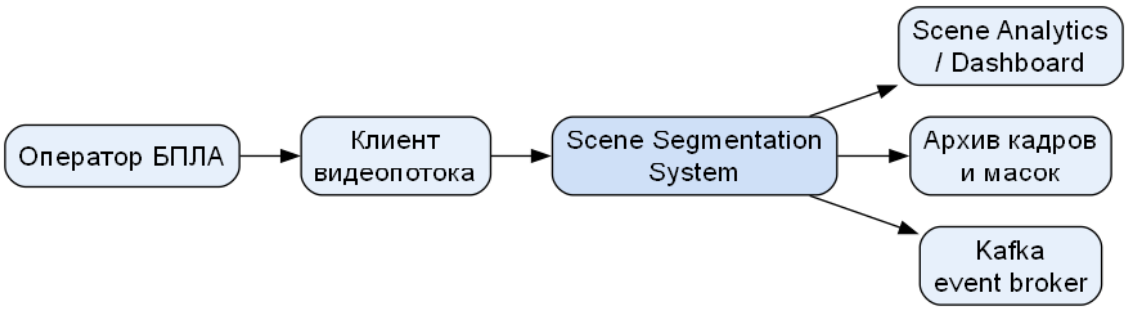


Рисунок 6 - C4 Context диаграмма системы сегментации

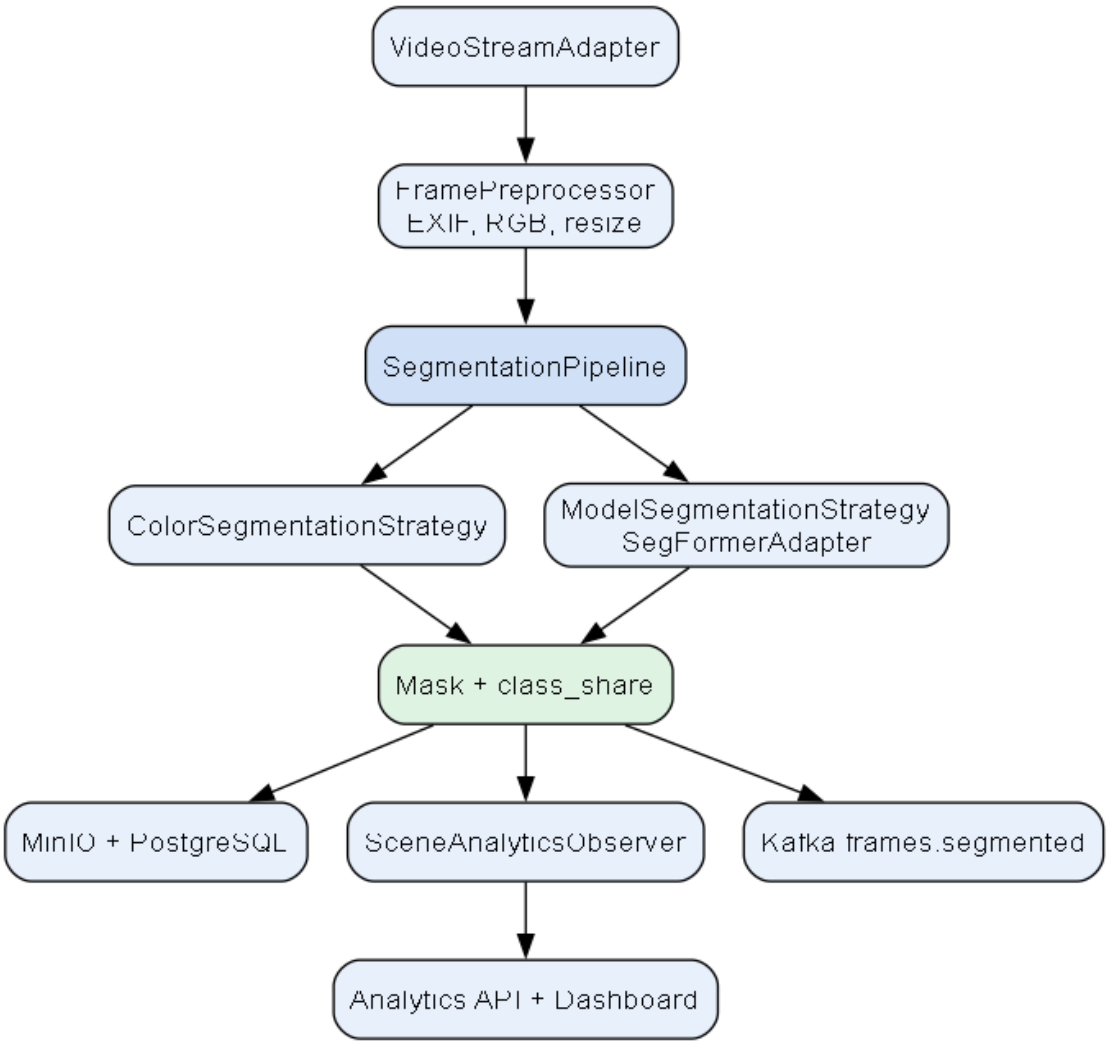


Рисунок 7 - C4 Component диаграмма системы сегментации

Таблица 5 - Сервисы и границы ответственности

Сервис/компонент	**Ответственность**	**Хранилище/контракт**
API	Принимает кадр, валидирует тело, возвращает результат и отдаёт dashboard endpoints.	FastAPI; POST/GET/summary/analytics
VideoStreamAdapter	Читает MP4, выделяет упорядоченные кадры и отправляет их в тот же pipeline.	OpenCV; frame_id prefix
Preprocessor	EXIF transpose, RGB-нормализация, ограничение 1280×720.	FramePreprocessor
SegmentationPipeline	Соединяет preprocessing, Strategy и формирование результата.	Pipeline pattern
Worker	Читает frames.created, получает объект MinIO, сегментирует и публикует frames.segmented.	Kafka group scene-segmentation
SceneAnalytics	Записывает размеры, dominant_class и доли классов для dashboard.	SQLite scene_analytics / PostgreSQL scene_analytics
MinIO/PostgreSQL	Сохраняют бинарные изображения/маски и production-метаданные.	frames/{id}.png, masks/{id}.png

7 Preprocessing, video stream и контракты сообщений

Каждый входной кадр сначала проходит EXIF-ориентацию, переводится в RGB и уменьшается с сохранением пропорций до максимума 1280×720. Это одинаково работает для HTTP-кадра и кадров, извлечённых из MP4. Сервис добавляет в ответ параметры preprocessing, чтобы результат можно было воспроизвести.

```
POST /api/v1/frames
X-Frame-Id: f2-0001
X-Flight-Id: flight1
```

```
Content-Type: image/png
Ответ 201:
{
  "frame_id": "f2-0001", "width": 1280, "height": 720,
  "dominant_class": "road",
  "class_share": {"road": 0.61, "pedestrian": 0.03},
  "preprocessing": {"orientation": "EXIF applied", "color_space": "RGB",
"max_size": [1280, 720]}
}
```

Kafka использует UTF-8 JSON и ключ frame_id. Producer публикует frames.segmented после сохранения маски и метаданных; worker является consumer группы scene-segmentation для frames.created. Полные схемы обязательных полей находятся в contracts/frames.created.json и contracts/frames.segmented.json.

8 Scene analytics и dashboard

SceneAnalyticsObserver записывает каждый обработанный кадр в таблицу scene_analytics. Индекс flight_id/captured_at ускоряет выборку временной линии полёта, индекс dominant_class в SQLite помогает группировке классов. Dashboard показывает количество кадров, используемое хранилище, режим Kafka, распределение доминирующих классов и последние кадры. В локальном режиме проверки Dashboard Kafka не запускалась. Отдельная проверка контейнерного режима с Kafka выполнена с использованием Docker Compose; работоспособность Kafka и обмен событиями подтверждены в ходе интеграционного тестирования.

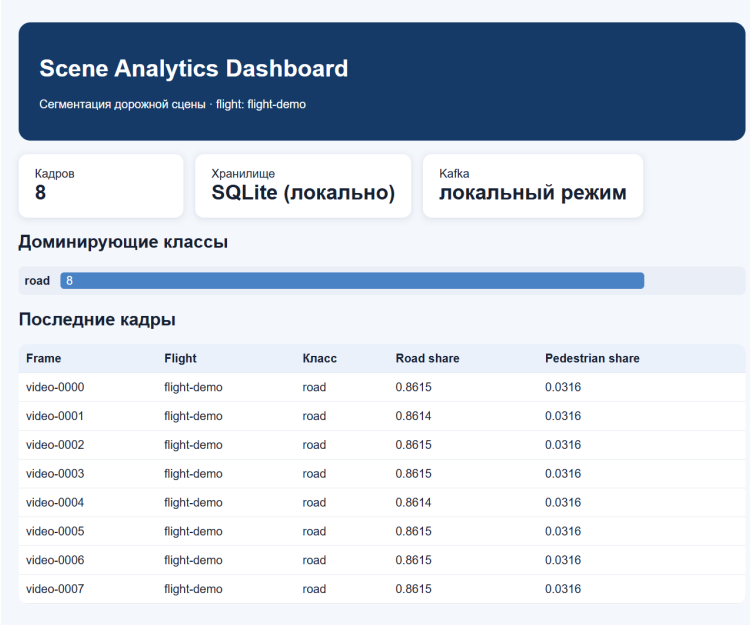


Рисунок 8 - Dashboard аналитики по flight-demo

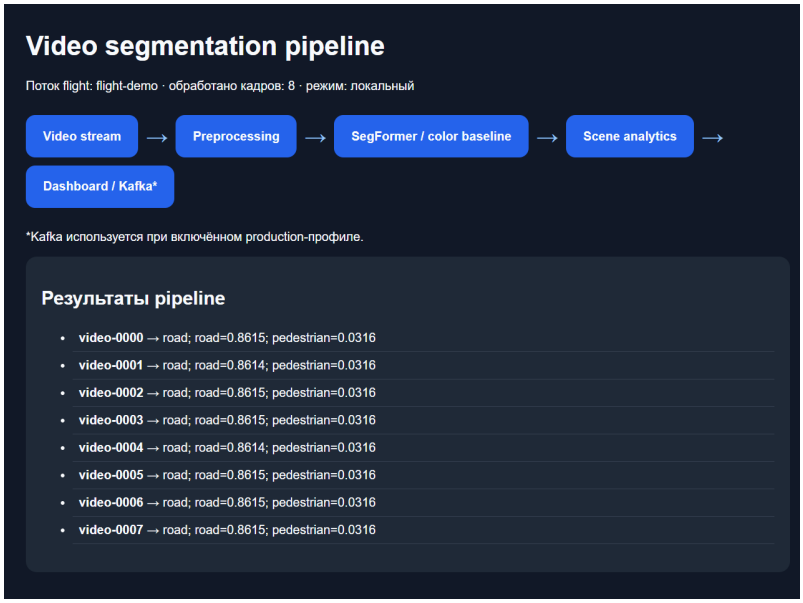


Рисунок 9 - Визуальный контроль video segmentation pipeline

9 ML-оценка, UAVid и границы применимости

Основной эксперимент использует comma10k: изображения и пиксельные маски автомобильной камеры. Предобработка модели включает RGB-конвертацию, processor Hugging Face и интерполяцию logits к размеру кадра. SegFormer b0 с Cityscapes сравнивается с цветовым baseline и OpenCV-поиском разметки. Метрики: per-class IoU, mean image IoU и pooled IoU. Дополнительная независимая проверка выполнена на 100 кадрах; результаты, имена файлов и SHA-256 записаны в artifacts/independent_metrics.json.

Для целевой аэровыборки добавлен отдельный скрипт evaluate_uavid.py. UAVid содержит UAV-последовательности и восемь классов, включая road, static car, moving car и human; набор распространяется правообладателем под CC BY-NC-SA 4.0. Файлы UAVid в комплект не включены. После размещения данных скрипт рассчитывает отдельные IoU для дороги, автомобилей и людей. Разметка не содержит отдельного класса дорожной разметки или универсального препятствия. Фактический UAV-прогон не выполнен; метрики comma10k не подтверждают качество на кадрах БПЛА.

10 Тестирование и распределение работы

Локальная проверка после доработки: 9 тестов unittest, включая preprocessing, pedestrian mask, безопасный frame_id, аналитику/dashboard, соответствие цветов UAVid и обработку упорядоченного MP4-потока; все 9

тестов завершились успешно. Дополнительно выполнены app.py --evaluate, --demo, --integration-status, video_pipeline.py --demo и docker compose config. Production-профиль Docker проверен 29.10.2026 на свободном host-порту 18012: API, worker, Kafka, MinIO и PostgreSQL перешли в состояние готовности и успешно прошли проверки работоспособности; кадр docker-audit-01 прошёл через API, запись появилась в PostgreSQL, исходный кадр и маска сохранились в MinIO, событие frames.segmented получено из Kafka. Результаты сохранены в artifacts/docker_ps.txt, docker_postgres.txt, docker_minio.txt и docker_kafka.txt.

Распределение работы: постановка задачи, проектирование архитектуры и контрактов, разработка, тестирование и подготовка отчёта выполняются исполнителем (то есть мной)

11 Контейнерная проверка production-профиля

Сборка production-профиля выполнена через Docker Compose. Сервисы API, worker, Kafka, MinIO и PostgreSQL запущены и получили статус healthy. Для исключения конфликта с занятым host-портом 8012 использовано временное сопоставление 18012:8012; внутри контейнера приложение продолжает слушать 8012. Проверочный POST /api/v1/frames вернул 201 и production-поля source_object, mask_object, postgres и kafka_topic.

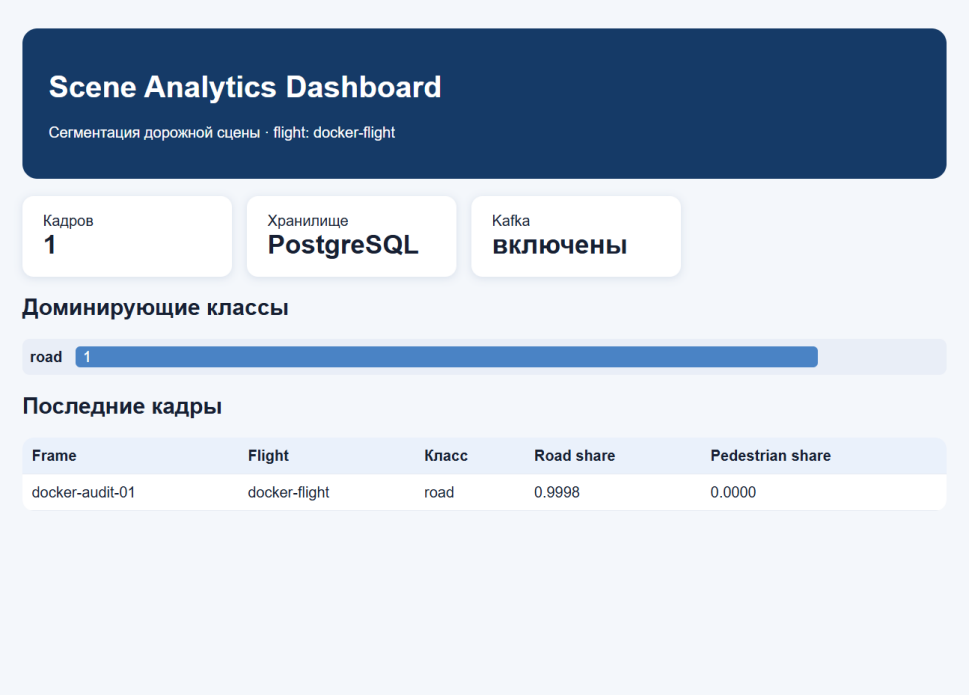


Рисунок 10 - Dashboard после контейнерного прогона с PostgreSQL и Kafka

Контрольные команды и результаты:

```
    docker compose -f services.yml config
docker compose -f services.check.yml up --build -d --wait
docker compose -p lr2scenecheck -f services.check.yml ps
GET /health -> 200
POST /api/v1/frames -> 201
PostgreSQL scene_analytics -> docker-audit-01
MinIO -> frames/docker-audit-01.png и masks/docker-audit-01.png
Kafka frames.segmented -> docker-audit-01
```

Полные текстовые логи контейнерной проверки находятся в artifacts/docker_ps.txt, artifacts/docker_postgres.txt, artifacts/docker_minio.txt и artifacts/docker_kafka.txt. После проверки контейнеры остановлены командой docker compose -p lr2scenecheck -f services.check.yml down; volumes проекта не удалялись.

Заключение

Разработан конвейер приёма кадров и семантической сегментации дорожной сцены с воспроизводимым цветовым baseline и адаптером SegFormer.

Список литературных источников

1. Apache Kafka Documentation [Электронный ресурс]. URL: https://kafka.apache.org/documentation/ (дата обращения: 26.10.2026).

2. Docker Docs [Электронный ресурс]. URL: https://docs.docker.com/ (дата обращения: 26.10.2026).

3. FastAPI Documentation [Электронный ресурс]. URL: https://fastapi.tiangolo.com/ (дата обращения: 26.10.2026).

4. Hugging Face Documentation [Электронный ресурс]. URL: https://huggingface.co/docs (дата обращения: 26.10.2026).

5. MinIO Documentation [Электронный ресурс]. URL: https://min.io/docs (дата обращения: 26.10.2026).

6. PyTorch Documentation [Электронный ресурс]. URL: https://pytorch.org/docs/ (дата обращения: 26.10.2026).

ПРИЛОЖЕНИЕ А. Результаты проверки

Таблица А.1 - Результаты и ограничения проверки

Показатель	**Содержание**
Локально подтверждено	9 модульных тестов; IoU дороги 0,7667 на 100 автомобильных кадрах
Production-контур в исходниках	PyTorch/OpenCV/SegFormer, Kafka, MinIO adapters; services.yml
Граница проверки	Перенос на БПЛА и отдельная оценка реальных пешеходов не подтверждены.

ПРИЛОЖЕНИЕ Б. Дополнительные результаты

Сегментация. В дополнительной оценке использовано 100 кадров comma10k, не входивших в первую десятку при подборе параметров. Среднее IoU итоговой маски приложения: дорога 0,7667, разметка 0,3198, движущиеся объекты 0,3576. Исходный класс дороги SegFormer - 0,7521 против 0,2912 у цветового baseline. Метка движущихся объектов объединяет автомобили, людей и животных. Файлы, хеши и параметры записаны в independent_metrics.json. Камера автомобильная; независимость кадров по поездкам и пересечение с обучением модели не установлены.

Сегментация: test_pedestrian_class_in_generated_frame проверил сохранение и маску на синтетическом кадре 10×10 с 10 пикселями пешехода; class_share pedestrian = 0,1. Это проверка цветового baseline и контракта, а не оценка SegFormer по реальным кадрам БПЛА.

Перенос сегментации на БПЛА остаётся неподтверждённым: включённая comma10k снята с автомобиля, а её метка движущихся объектов не разделяет пешеходов и машины. Для оценки качества на БПЛА необходима независимая размеченная аэровыборка. Успешные отдельные тесты и smoke-прогон не подтверждают качество на всех данных.

Источник данных: https://github.com/commaai/comma10k

ПРИЛОЖЕНИЕ В. Развёртывание

Dockerfile: Сегментация дорожной сцены

```
FROM python:3.12-slim
WORKDIR /app
ENV PIP_DEFAULT_TIMEOUT=180 PIP_RETRIES=5
RUN pip install --no-cache-dir --extra-index-url
https://download.pytorch.org/whl/cpu torch==2.14.1+cpu
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8012
CMD ["uvicorn", "api:api", "--host", "0.0.0.0", "--port", "8012"]
```

Порядок запуска в PowerShell

Откройте PowerShell в папке проекта, запустите Docker Desktop и выполните команды ниже. После запуска доступен адрес http://localhost:8012; для проектов с FastAPI документация доступна по адресу /docs. Подробности о локальном запуске и моделях приведены в README.md.

```
docker compose -f services.yml config
docker compose -f services.yml up --build -d --wait
docker compose -f services.yml ps
python -X utf8 -m unittest -v
docker compose -f services.yml down
```